Newton School of Technology

18

The Foundations of Agentic AI

From Chatbots to Digital Interns: Moving beyond simple conversation to autonomous action in artificial intelligence systems.

Newton School of Technology

PART 1

The Foundations of Agentic AI

From Chatbots to Digital Interns

Moving beyond "Talk" to "Action" | The AI as your Digital Intern

1.1 The Digital Intern Analogy

Newton School of Technology

The Traditional Chatbot

A standard LLM (like ChatGPT) is like a brilliant encyclopedia —it can answer questions, generate text, and engage in conversation, but it doesn't "do" anything beyond providing information.

- Limited to conversation only

Standard LLM

The Encyclopedia

- Answers questions
- Generates text
- Conversational only

1.1 The Digital Intern Analogy

Newton School of Technology

The Digital Intern (Agentic AI)

Agentic AI is different. It is a sophisticated system that goes beyond simple chat. Like a capable intern, it can reason, plan, and execute tasks using available tools.

- Reasons about complex problems
- Uses tools to interact with the world
- Follows instructions to complete workflows

Agentic AI

The Digital Intern

- Reasons & plans
- Uses tools
- Executes tasks

The Analogy: An intern has general knowledge (the Brain), but to be useful, they need specific instructions on what to do, and access to company software (Tools) to execute tasks.

The Anatomy of an Agent

Newton School of Technology

PILLAR 1
The Brain

The LLM

The engine of reasoning, language understanding, and decision-making. The Brain provides the "common sense" and analytical capabilities that figure out how to solve a problem.

- Reasoning & logic
- Language comprehension
- Decision-making

Newton School of Technology

PILLAR 2
The Hands

Tools

Capabilities that allow the AI to interact with the outside world. An intern without hands cannot file paperwork. Tools let the agent actually perform tasks and manipulate external systems.

- Searching files
- Running code
- Accessing APIs

Newton School of Technology

PILLAR 3
The Instructions

Prompts

The logic defining the agent's purpose, behavior, and workflow. As the manager, you must define structured instructions that tell the brain how to use the hands to achieve goals.

- Agent purpose
- Behavior rules
- Workflow logic

Integration: These three pillars work as an integrated system. The Brain reasons about the task, the Instructions guide the approach, and the Hands execute actions in the real world.

1.3 Deep Dive into the Brain (LLMs)

Understanding Large Language Models as Probabilistic Prediction Engines

Newton School of Technology

What is an LLM?

A probabilistic prediction engine, not a sentient mind. LLMs function as sophisticated "next-word predictors" that calculate the probability of which word comes next based on patterns learned from massive text datasets.

1.3 Deep Dive into the Brain (LLMs)

Understanding Large Language Models as Probabilistic Prediction Engines

Newton School of Technology

① Training Phase

Modeling Phase: The model learns to minimize prediction error on massive text datasets, absorbing patterns, grammar, facts, and reasoning structures.

🗄 Billions of text examples → Pattern recognition

1.3 Deep Dive into the Brain (LLMs)

Understanding Large Language Models as Probabilistic Prediction Engines

Newton School of Technology

② Decoding Phase

Generation Phase: When generating text, the model produces a "vector of probabilities" for the next token (word/part of word) based on statistical likelihood.

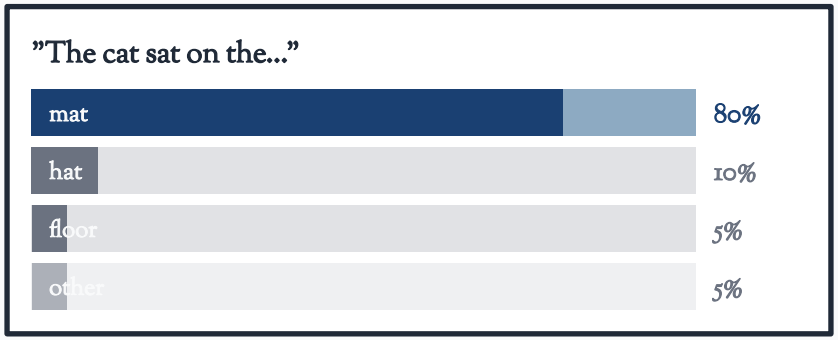
Probability distribution → Token selection

Decoding Strategies

Whether the AI chooses the most likely word **(Greedy Search)** or gets creative **(Stochastic Sampling)** depends on configuration parameters like temperature and top-p.

1.3 Deep Dive into the Brain (LLMs)

Newton School of Technology



Key Insight: The model generates a probability distribution over all possible next tokens, then samples from this distribution based on the decoding strategy.

⚠ Critical Understanding

It doesn't "know" the answer; it predicts the most likely continuation of your text based on statistical patterns learned during training.

1.4 Closed vs. Open Source Brains

Choosing Between Proprietary and Open-Weight Language Models

Newton School of Technology

The Walled Garden

Closed Source / Proprietary

Examples

GPT-4/4.5 | Gemini 1.5/2.5 | Claude 4

Pros

- Highest reasoning benchmarks
- Managed infrastructure (no setup)
- Regular updates & improvements

Cons

- Paid API / Subscription required
- Data leaves your network
- "Black Box" (unclear mechanics)

1.4 Closed vs. Open Source Brains

Choosing Between Proprietary and Open-Weight Language Models

Newton School of Technology

The Community Park

Open Source / Open Weights

Examples

Llama 4 | Gemma 3 | Mistral Large 3

Pros

- Free to run (no API fees)
- Privacy-focused (local execution)
- Customizable & transparent

Cons

- Requires local hardware (RAM/GPU)
- Manual setup & configuration
- Performance varies by model size

Market Trend: While closed models often lead benchmarks, open models like Llama 4 and Mistral Large 3 are rapidly closing the gap, offering near-proprietary performance with complete control and privacy.

1.5 Why Go Open Source?

The Strategic Advantages of Local LLM Deployment

Newton School of Technology

ADVANTAGE 1

Complete Data Privacy

Your prompts and proprietary data never leave your device. When analyzing internal financial documents, patient records, or confidential research, local execution ensures complete data sovereignty.

- No third-party data processing

ADVANTAGE 2

Security & Compliance

Essential for healthcare (HIPAA), finance, and enterprise secrets. Regulatory frameworks often prohibit transmitting sensitive data to external cloud services. Local LLMs ensure compliance.

- Meet regulatory requirements

1.5 Why Go Open Source?

The Strategic Advantages of Local LLM Deployment

Newton School of Technology

ADVANTAGE 3
Offline Capability

Run agents on an airplane, in a secure facility, or without internet reliance. Once downloaded, open-source models operate entirely offline, enabling AI capabilities in disconnected environments.

- Zero internet dependency

ADVANTAGE 4
Cost Control

No "per-token" API fees or monthly subscriptions. Once you have the hardware, the model is free to use forever. Eliminate unpredictable cloud AI costs and budget with certainty.

- Predictable fixed costs

Bottom Line: Open-source LLMs provide enterprise-grade AI capabilities with complete control over data, compliance, connectivity, and costs.

1.6 Introducing the Brain Manager (Ollama)

The "MP3 Player" for AI Models

Newton School of Technology

What is Ollama?

The "MP3 Player" for AI Models. Just as an MP3 player takes a music file and plays it, Ollama takes a raw AI model file and makes it usable.

Raw AI models are heavy files full of numbers (weights). You can't just "open" them like a document. Ollama acts as the backend "runner" that manages the model.

Role & Function

A **lightweight runtime** that allows you to easily set up and run LLMs locally. It manages hardware resources (CPU/GPU) so you can focus on talking to the agent.

- Manages CPU/GPU allocation
- Handles model loading/unloading
- Provides API interface

1.6 Introducing the Brain Manager (Ollama)

The "MP3 Player" for AI Models

Newton School of Technology

The Ollama Analogy

AI Model File
(e.g. Llama-4.gguf)
Raw weights & parameters

+

Ollama Runtime
The "MP3 Player"
Model executor & manager

=

Working AI Agent
Interactive Chat
Ready for conversation

Why Ollama?

- ✓ **Simplified Setup**
  No manual Python environment
- ✓ **Model Library**
  Llama, Gemma, Mistral, Qwen
- ✓ **Dual Interface**
  CLI & API support
- ✓ **Cross-Platform**
  macOS, Linux, Windows

1.7 Master the Ollama CLI

Essential Command-Line Interface Operations

Newton School of Technology

1 ollama pull [model_name]

Action: Downloads the model "brain" to your machine.

```
ollama pull llama4
```

Like downloading an app—fetches the model files from the registry.

2 ollama list (no arguments)

Action: Shows all "brains" currently stored in your library.

```
ollama list
```

Checks your inventory: "What models do I have installed?"

3 ollama run [model_name]

Action: Loads the model into memory and starts the chat session.

```
ollama run llama4
```

The main event—boots up the agent. Auto-pulls if model missing.

4 ollama rm [model_name]

Action: Deletes the model to free up disk space.

```
ollama rm llama4
```

Models are large (GBs). Use this to clean up when done.

Pro Tip

Use `ollama ps` to check which models are currently active and consuming RAM—like Task Manager for AI.

1.8 Interactive Lab: Command Matching

Match the Ollama CLI Command to the Real-World Scenario

Newton School of Technology

A

Scenario

"I want to download Llama 4 without starting it yet."

Correct Command:
ollama pull llama4

B

Scenario

"My hard drive is full! I need to remove unused models."

Correct Command:
ollama rm [model]

C

Scenario

"Let's ask the AI a question about Python programming."

Correct Command:
ollama run [model]

D

Scenario

"Do I already have Gemma 3 installed on this machine?"

Correct Command:
ollama list

Additional Pro Tip

To check which models are currently active and consuming RAM (similar to checking Task Manager), use the command `ollama ps`. This shows running models, their sizes, and processor usage.

Newton School of Technology

Building Your First Agent

You now understand the foundations.

The Digital Intern awaits your instructions.

The Brain

LLM reasoning & decision-making

The Hands

Tools for external interaction

The Instructions

Prompts defining purpose & workflow

→ **Goal: Moving from "Chatting" to "Doing"**

The Limits of the Brain

Why a "Naked" LLM Fails

Newton School of Technology

Training Cutoffs

The model only knows what it learned during training. It cannot access real-time information like today's weather or current stock prices.

Hallucinations

LLMs predict the next word, not verify facts. When uncertain, they generate plausible-sounding but potentially false information.

No Actions

A text model cannot click "Send" on an email, query a database, or interact with external systems. It only outputs text.

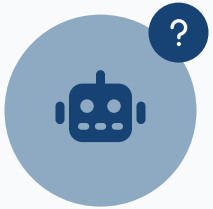
Poor Math

LLMs are linguistic engines, not computational ones. They struggle with precise mathematical calculations and logical operations.

The Limits of the Brain

Why a "Naked" LLM Fails

Newton School of Technology



EXAMPLE 1

Q: "What's the weather in Tokyo right now?"

A: [Hallucinates a realistic but fake answer]

EXAMPLE 2

Q: "Calculate 2347 × 8563"

A: [Approximate, often incorrect result]

The model isn't lying—it's predicting tokens without access to external data or computational tools.

Introducing Tools (The Hands)

External Functions for Agent Capabilities

Newton School of Technology

What are Tools?

1. External functions the agent can **"call"** when it needs help beyond its training data.
2. The **Brain (LLM)** decides which tool to use, and the **System** executes it.
3. This is the foundation of **"Agentic AI"** and **Function Calling**.

Web Search

Access real-time information from the internet. Retrieve current news, facts, and data beyond training cutoffs.

Real-time | Dynamic

Calculator

Solve mathematical problems with precision. Perform complex calculations accurately instead of token prediction.

Precise | Reliable

API Requests

Connect to external services and databases. Fetch weather, stock prices, send emails, or query enterprise systems.

Connected | Versatile

Code Interpreter

Execute code to process data, generate visualizations, or perform complex data analysis tasks programmatically.

Executable | Powerful

Key Concept: The LLM generates a specific command to invoke a tool when it realizes it cannot answer from training data alone. This is called **Function Calling** or **Tool Calling**.

The Agentic Workflow

The Loop of Reasoning

Newton School of Technology

User Request
"Is it raining in London?"

→

LLM Reasoning
Analyzes request
"Need real-time data"

→

Tool Selection
Chooses
Weather_Tool

→

Execution
Fetches data
"Rainy, 12°C"

→

Synthesis
"Yes, it is raining in London"

How the Loop Works

1. **User Request**
   The user submits a question or task that requires information beyond the LLM's training data.
2. **Reasoning & Analysis**
   The LLM analyzes the request and recognizes it needs external data or capabilities.
3. **Tool Selection**
   The LLM selects the appropriate tool from its available toolkit and generates the tool call.
4. **System Execution**
   The system executes the tool, fetches the data, and returns the raw results to the LLM.
5. **Synthesis & Response**
   The LLM synthesizes the tool output into a natural language response for the user.

Technical Note: The LLM pauses generation to wait for tool output. Data flows out and back in before completion.

Setting Up the Builder (n8n)

Workflow Automation for AI Agents

Newton School of Technology

What is n8n?

Fair-code workflow automation platform

A visual, node-based workflow automation tool that integrates seamlessly with Ollama and other AI services. Build complex agent workflows without writing code.

- Visual
- Self-Hosted
- Private

Installation Options

Self-Hosted (Recommended)

Keep your data private and local. Run n8n on your own machine with full control.

Terminal Command:

```
npx n8n
```

Requires Node.js installed

Cloud (n8n.io)

Quick start without local setup. Create an account and start building immediately.

Website:
https://n8n.io

Connection to Ollama

Ollama Server
http://localhost:11434

n8n Workflow Builder
Visual node interface

Important: Ensure your Ollama server is running in the background (`ollama serve`) before starting n8n.

Workshop: The Weather Reporter Agent

Building Your First Functional Agent

Newton School of Technology

n8n Canvas Workflow

1. Chat Trigger

The interface where you type messages. Captures user input and starts the workflow.

Input Node | Webhook

↓

2. AI Agent (Basic LLM Chain)

The reasoning engine connected to Ollama.

- ✓ Model: Llama 3 or Mistral
- ✓ Prompt: "Use tools when necessary"
- ✓ Memory: Window Buffer attached

↓

3. Weather Tool (HTTP Request)

Connects to weather API (e.g., OpenWeatherMap) to fetch real-time data.

API Integration | Real-time Data

Key Configuration Steps

1. **Drag Chat Trigger**
   Represents the user interface
2. **Add Basic LLM Chain**
   Select 'Ollama' as Model Provider
3. **Attach Memory Node**
   Connect Window Buffer Memory
4. **Define Tool**
   Add Weather API as available tool

Demo Tip

For initial testing, simulate a weather tool that always returns "Sunny, 22°C" to verify the agent correctly picks up and uses the tool.

Live Demo
Switch to n8n Canvas

Data Flow Verification

Tracing the Journey of Information

Newton School of Technology

The Data Journey

Input Stage

User text enters the chain as a JSON object with the query.

```
{ "input": "Weather in NY?" }
```

Tool Call Stage

LLM spots intent and pauses to request data via tool invocation.

```
{ "tool": "get_weather", "location": "NY" }
```

Output Stage

Tool provides raw JSON data, LLM translates to natural language.

```
"It is currently raining in New York."
```

Execution View in n8n

Use the **"Execution" view** to see the data flow happening in real-time. Don't just trust the agent works—verify the data flow.

- Monitor each node's input and output
- See the exact moment the model stops writing text
- Verify the tool command is generated correctly
- Confirm data returns to the LLM for synthesis

What to Look For

1. JSON structure in each node
2. Tool invocation parameters
3. Response data from external APIs
4. Final synthesized output

Verification Principle: You will actually see the moment the model stops generating text and starts writing a computer command to fetch the weather. This is the hallmark of true agentic behavior.

Memory & Context

The "Notepad" for Conversational Agents

Newton School of Technology

The Problem

LLMs are Stateless

Large Language Models are **stateless**—they forget you immediately after generating a response. Each interaction is independent.

WITHOUT MEMORY

You: "What's the weather in Paris?"

Agent: "It's sunny in Paris."

You: "What about London?"

Agent: "What about London what?"

The Solution

Window Buffer Memory

Attach a **Memory Node** that acts as a "Session Notepad." It records the conversation and sends previous turns back to the brain.

WITH MEMORY

You: "What's the weather in Paris?"

Agent: "It's sunny in Paris."

You: "What about London?"

Agent: "In London, it's rainy."

Memory & Context

The "Notepad" for Conversational Agents

Newton School of Technology

How Memory Works

1. **Conversation Recording -** Memory node stores each exchange (user message + agent response) in a buffer.
2. **Context Injection -** With each new message, the entire conversation history is re-sent to the LLM along with the new query.
3. **Window Management -** Configure how many previous turns to remember (e.g., last 5-10 exchanges) to manage token usage.

Context Window - The **Context Window** is the maximum amount of text (tokens) the model can process in a single request.

Llama 3 (8B)	8K tokens
Llama 3 (70B)	128K tokens
Mistral	32K tokens

In n8n: Connect the Window Buffer Memory node to the Memory input of the LLM Chain node.

Final Review & Summary

The Digital Intern Framework

Newton School of Technology

The Brain

REASONING

LLM via Ollama — the cognitive engine that analyzes requests, reasons about solutions, and synthesizes responses.

The Office

ENVIRONMENT

n8n (Workflow Builder) — the visual environment where brain, tools, and memory are orchestrated into functional workflows.

The Hands

ACTION

Tools (Search, API, Calculator) — external capabilities that extend the agent's reach beyond text generation.

The Notepad

CONTEXT

Memory (Window Buffer) — the session history that enables multi-turn conversations and contextual understanding.

What We Built

Transformed a raw prediction engine into an agent with hands, tools, and memory—capable of real-world actions.

Key Skills

Tool integration, workflow design, data flow verification, and memory management for conversational AI agents.

Thank You !!

See You Guys in Next Session :)